

Purpose and implementation

Problem and user. Berlin visitors and residents need a quick way to identify a restaurant and plan an evening. Our persona is an English-speaking visitor meeting a friend for dinner. The goal is restaurant discovery and transparent planning support, without accounts or booking complexity.

Product. Search live Google Places results, confirm a restaurant, select a Berlin-local visit time and read “Tonight at a Glance.” Cards separate provider facts, unknown information and a clearly labelled crowd heuristic. Save only place identifiers in local browser storage.

User flow and interface

Search → confirm restaurant → choose visit time → Tonight at a Glance → save or search again.

The interface uses cream, cherry red, large serif headings and rounded cards. It includes loading, empty, missing-key and provider-error states. No fabricated restaurants appear in the product. Visual browser verification is pending.

The current implementation includes searchable cuisine suggestions, explicit loading and error messages, live restaurant detail retrieval, Berlin-local visit time input, a saved-ID list and separate privacy/terms routes. Keyboard focus styles and reduced-motion styling are included. These controls must still be exercised in an actual browser at mobile and desktop widths.

API and adapters

/ Route / Purpose /

/ GET /api/health / Process health, no provider calls /

/ GET /api/config / Credential-presence booleans, never values /

/ GET /api/search?q=... / Live restaurant search /

/ GET /api/places/{id} / Live details /

/ POST /api/insight / { "place_id": "...", "visit_at": "ISO timestamp with offset" } /

/ GET /api/docs / OpenAPI interactive documentation /

The local rate limiter is per process and ignores untrusted forwarded headers. AWS WAF provides the distributed public-facing limit. Rate limits reduce abuse but do not guarantee a spending ceiling.

Data and estimation

1. Search by restaurant name or craving, within the Berlin bounding rectangle.
2. Select a result; Google details are fetched on demand.
3. Choose a Berlin-local visit time, up to seven days ahead.
4. Read crowd estimate, editorial quick take when available, language/booking uncertainty, current weather, and general advice.
5. Save a place ID locally and fetch fresh details when reopening it.

Google is the source for restaurant details. We request explicit field masks and show attribution. Editorial summaries are displayed as supplied; Google reviews and restaurant content are ****not** sent to OpenAI^{**}. The AI receives only our generic time heuristic. This is intentionally not an AI review summarizer. Staff English ability and reservation availability are not verified and are never inferred from location or translated reviews.

Crowd estimates are simple, unvalidated rules: base 25, +35 dinner (18–20:59), +20 lunch (12–13:59), +20 Friday/Saturday. Quiet <40, Moderate 40–64, Busy ≥65. These values express a heuristic, not measured occupancy, probabilities or a trained ML model. Weather is current weather and does not influence this score. Open-Meteo's free endpoint is for qualifying noncommercial use; revisit licensing before commercial operation.

Cloud architecture

Architecture. Browser → CloudFront HTTPS/WAF → Application Load Balancer → two FastAPI/Fargate tasks across two availability zones. ECR holds the portable image. Secrets Manager supplies credentials. CloudWatch records logs and alarms. Terraform describes the infrastructure. Both tasks access Google Places, Open-Meteo and optional OpenAI over HTTPS.

Rationale and alternatives. One Docker image runs locally or on AWS. Service adapters isolate providers. Standard ECS exposes architecture choices for the portfolio. App Runner was rejected after verifying its closure to new customers. Lambda and a separate S3 frontend remain alternatives. Omitting a database reduces cost but saved lists do not sync.

Best practices and limits. Two tasks, health checks and CPU scaling support availability. Secrets stay out of source. WAF limits traffic. Public-subnet tasks avoid NAT costs and accept inbound traffic only from the ALB. The CloudFront-to-ALB hop is HTTP in this draft; a domain and ACM certificate are needed for end-to-end TLS. Always-on resources incur cost.

Installation

Python 3.12+ is required. On macOS, from this directory:

```
python3 scripts/configure_keys.py  
bash start.command
```

The first command accepts both keys as hidden input. It writes `.env` with owner-only permissions. Never put keys in GitHub, chat, frontend JavaScript, screenshots or Terraform variables. The launcher creates a virtual environment, installs dependencies, runs tests, and starts the app at `**http://127.0.0.1:8000**`. A dependency or test failure stops startup so it can be fixed first.

Google Cloud: enable `**Places API (New)**`, billing and an API key restricted to that API. The key belongs to the backend. For OpenAI, use an API project key with billing and model access. `OPENAI_MODEL` is configurable. API billing is separate from a ChatGPT subscription.

If no Google key is present, searches return an explicit 503 setup message. The app never fabricates restaurant results. Without an OpenAI key, factual Google search still works; the app labels the fixed general advice instead of claiming AI generation. Restart after changing `.env`.

Docker — the same app

```
cp .env.example .env # only if .env does not already exist  
python3 scripts/configure_keys.py  
docker compose up --build
```

Open `http://127.0.0.1:8000`. The image runs as a non-root user; Compose limits host exposure to localhost, drops capabilities and uses a read-only filesystem. `.dockerignore` excludes credentials and Terraform state.

Deployment procedure

Read docs/DEPLOYMENT.md before creating resources. Install Terraform providers and validate the configuration, then review the bootstrap plan and calculate current regional costs. Obtain approval before applying.

Bootstrap creates infrastructure with `deploy_app=false`. Store both keys privately in Secrets Manager using the hidden-input script. Build an amd64 image and push it to ECR with an immutable tag. Review an application plan with `deploy_app=true` before applying it. Record the resulting CloudFront URL only after the service is stable.

The current IAM identity has authentication permission only. A deployment role needs scoped permissions for the configured services and restricted PassRole access. The user must approve that access. Runtime tasks receive only the execution permissions needed for logs, image retrieval and the two secrets.

Verification record

Validation record

Date: 2026-09-24. No AWS deployment or live-data success is implied.

/ Check / Result /

/ Python syntax compilation / Passed /

/ Frontend JavaScript syntax / Passed /

/ Automated API, adapter and scoring tests / 12 passed; one upstream TestClient deprecation warning /

/ Local server / Started successfully by the user on port 8000 /

/ Terraform HCL formatting / Passed /

/ Terraform init and provider validation / Passed with AWS 6.66.0 and random 3.9.1 /

/ Docker builds / Passed for local arm64 and AWS-targeted linux/amd64 /

/ Docker runtime / Health and homepage respond; UID 10001; no /app/.env; missing-key search returns 503; linux/amd64 smoke test also passed /

/ Visual/interaction review / User has the local app open; agent visual review not performed /

/ Live Google/OpenAI / Not tested: no keys provided /

/ AWS identity / Verified as berlinbite-dev /

/ AWS plan / Incomplete: two EC2 Describe permissions denied; no resources created /

/ GitHub publication / Direct Git push succeeded; app connector separately returns 403 /

The initial authoring restrictions blocked package downloads and Docker. The user installed local dependencies; subsequent approved command access enabled container and Terraform validation. The final results above supersede those initial blockers.

Complete the remaining checks before marking the app or portfolio submission-ready. Never replace missing deployment evidence with a diagram or pretend that a mock test is a live provider call.

Operations and limitations

Monitor target 5xx errors, ECS task health and CloudWatch logs. The alarm has no notification destination yet. Do not log secrets or upstream response bodies. WAF rate limits reduce abuse but do not set a spending ceiling.

Rotate keys in Secrets Manager and restart tasks to load them. Roll back by selecting a previous immutable image tag. Review destruction plans deliberately; ECR images and recoverable secrets require careful cleanup. Protect Terraform state and migrate it to an encrypted remote backend for shared work.

Before public use: finish operator privacy details, configure provider quotas and costs, complete live tests and consider origin TLS. No uptime, load, latency or security certification has been measured.

Reflection and references

App Runner's new-customer closure changed the hosting choice. Standard ECS offers explicit infrastructure; Express Mode could simplify a later deployment.

The single container reduces migration work. The heuristic is useful only as labelled generic guidance. Omitting login/database reduces scope but limits saved-list convenience. Highest priority is completing real execution checks.

Future research could collect appropriately licensed occupancy observations, define labels, compare a trained model to the transparent time baseline and evaluate calibration and error. This requires a defensible dataset; it cannot be replaced by invented crowd data.

References: Google Places API documentation and policies; FastAPI documentation; Docker documentation; HashiCorp Terraform documentation; AWS ECS/Fargate and App Runner availability documentation; Open-Meteo documentation; OpenAI Responses API documentation. Direct links are supplied in README.md.

This is a development draft, not a claim of completed deployment. Phase 1/2 and abstract drafts accompany it. The student must replace pending evidence and add genuine personal reflection and feedback before final submission.